RRM — Release Readiness Module

RRM — Release Readiness Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: SRLS — Software Release Standard

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Release Readiness Governance

Module ID: SLGA-SRLS-RRM-02

Origin ID: OOF-SLGA-SRLS-RRM

Version: 1.0

Status: Canonical Open

Effective Date: 13.10.2026

Compatibility: Methodology OS — Modular Compatibility

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: Protected Methodological Work

Canonical Language: English

Canonical Definition

RRM — Release Readiness Module is the Core Module within SRLS that governs how an identifiable software release candidate is assessed against applicable requirements, quality, security, dependency, configuration, evidence, and release-specific conditions to determine whether its readiness for release authorization is sufficiently established.

RRM establishes the methodological conditions under which a software

release candidate may be recognized as ready, not ready, conditionally ready, evidence-limited, conflicted, or undetermined within an explicitly defined release scope.

It governs the relationship between the exact release candidate, applicable

readiness criteria, supporting lifecycle assessments, unresolved findings, exceptions, restrictions, and the resulting readiness determination.

RRM ensures that successful builds, completed development tasks, passing tests, automated pipeline results, or agent-generated readiness declarations cannot independently establish software release readiness.

The module preserves the distinction between technical verification results

and the broader governance conditions required to support a release decision.

It also ensures that readiness conclusions remain bound to the candidate identity, assessment boundary, evidence, and conditions under which they were established.

RRM does not independently grant release authorization or deployment permission.

Its principal governed output is the Software Release Readiness Profile.

Build Success ≠ Release Readiness.

Testing Success ≠ Sufficient Release Verification.

Release Conditions Assessed ≠ Release Conditions Satisfied.

Release Readiness Established ≠ Release Authorization Granted.

Release Readiness Valid Previously ≠ Release Readiness Valid After

Material Change.

Operational Role

RRM governs the evidence-supported assessment of software release readiness.

Its operational role is to establish:

• Release candidate identity

• Readiness assessment identity

• Assessment purpose

• Assessment scope and boundary

• Applicable release criteria

• Requirements alignment

• Quality condition references

• Security condition references

• Dependency condition references

• Configuration condition references

• Artifact integrity references

• Test coverage relevance

• Unresolved defects

• Unresolved vulnerabilities

• Release restrictions

• Evidence provenance

• Evidence sufficiency

• Evidence applicability

• Evidence conflicts

• Readiness uncertainty

• Condition status

• Exception applicability

• Material finding assessment

• Readiness classification

• Reassessment triggers

• Readiness validity boundaries

• Authorization handoff

RRM evaluates the applicability of findings established by responsible

lifecycle standards.

It does not replace their underlying technical assessments, independently certify software safety, or authorize the release.

Operational Question

How can an identifiable software release candidate be assessed for release readiness when requirements, quality, security, dependencies, configurations, testing evidence, unresolved findings, and automated development results may differ in completeness, reliability, and applicability, while ensuring that unknown conditions, exceptions, material changes, and authorization boundaries remain explicitly governed?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

SRLS Core Module Flow

RCMM — Release Candidate Management Module

2. RRM — Release Readiness Module 3. RAM — Release Authorization Module 4. RRMM — Release Record Management Module 5. RWM — Release Withdrawal Module

RRM occupies the second Core Module position within SRLS.

It receives the Software Release Candidate Profile established by RCMM.

It evaluates the candidate against applicable release conditions and

supporting lifecycle governance references.

Its output establishes the readiness state submitted to RAM for release

authorization consideration.

RRM does not convert a readiness determination into an authorization decision.

Software Release Candidate Profile → Readiness Assessment Scope →

Applicable Release Criteria → Requirements Alignment → Quality & Security Conditions → Dependency & Configuration Conditions → Evidence Sufficiency → Unresolved Findings → Exception Assessment → Readiness Classification → Software Release Readiness Profile → RAM Authorization Handoff

• Governed Space

Software Release Readiness Governance

This governed space concerns the evidence-supported determination of

whether an identifiable software release candidate satisfies the applicable conditions for consideration under release authorization governance.

It includes:

• Readiness assessment identity

• Candidate association

• Assessment scope

• Readiness criteria

• Requirements alignment

• Quality findings

• Security findings

• Dependency conditions

• Configuration conditions

• Artifact integrity conditions

• Evidence sufficiency

• Unresolved findings

• Materiality

• Exceptions and restrictions

• Readiness classification

• Validity boundaries

• Reassessment

• Authorization handoff

RRM does not independently govern release authorization, deployment authorization, production operation, or general authority legitimacy.

Governed Object

The primary Governed Object is:

A Software Release Readiness State representing the evidence-supported

relationship between an identifiable release candidate, applicable readiness criteria, relevant lifecycle findings, unresolved conditions, exceptions, restrictions, and the resulting readiness determination within defined assessment boundaries.

The governed state must preserve:

Release Candidate ID

Readiness Assessment ID

Assessment Purpose

Assessment Boundary

Applicable Readiness Criteria

Requirements Alignment State

Quality Condition State

Security Condition State

Dependency Condition State

Configuration Condition State

Artifact Integrity State

Evidence Sufficiency State

Unresolved Findings

Exception References

Readiness Classification

Validity Conditions

Reassessment Requirement

Version

Scope

RRM governs:

• Release readiness assessment establishment

• Candidate-specific readiness criteria

• Requirements alignment review

• Quality and security condition integration

• Dependency and configuration condition integration

• Artifact integrity references

• Test evidence applicability

• Unresolved finding assessment

• Evidence sufficiency

• Condition classification

• Exception and restriction treatment

• Readiness determination

• Readiness validity

• Reassessment

• Authorization handoff

RRM does not independently perform software development, testing, security remediation, release approval, or deployment.

Foundational Governance Principles 8.1 Readiness Principle

Build Success ≠ Release Readiness. 8.2 Verification Principle

Testing Success ≠ Sufficient Release Verification. 8.3 Requirements Principle

Implemented Behavior ≠ Approved Software Requirements. 8.4 Evidence Principle

Readiness Evidence Available ≠ Readiness Evidence Sufficient. 8.5 Condition Principle

Release Conditions Assessed ≠ Release Conditions Satisfied. 8.6 Uncertainty Principle

Unknown Release Condition ≠ Satisfied Release Condition. 8.7 Exception Principle

Release Exception Approved ≠ Underlying Condition Satisfied. 8.8 Authorization Principle

Release Readiness Established ≠ Release Authorization Granted. 8.9 Candidate Identity Principle

Readiness Established for One Candidate ≠ Readiness Established for

Another Candidate. 8.10 Continuity Principle

Release Readiness Valid Previously ≠ Release Readiness Valid After

Material Change.

Readiness Assessment Identity

Every governed readiness assessment must establish:

Readiness Assessment ID

Release Candidate ID

Software Product ID

Assessment Purpose

Assessment Scope

Assessment Boundary

Applicable Criteria Reference

Assessment Time

Evidence References

Assessment Classification

Version

Assessment Identity Principle

Readiness Assessment Recorded ≠ Readiness Assessment Applicable to the

Current Candidate.

The assessment must remain associated with the exact candidate state evaluated.

Readiness Assessment Purpose

The assessment purpose establishes why release readiness is being evaluated.

Relevant purposes include:

Initial Release Readiness

Maintenance Release Readiness

Security Patch Readiness

Emergency Release Readiness

Candidate Reassessment

Restricted Release Readiness

Post-Change Readiness Review

The assessment purpose does not independently determine which mandatory conditions may be omitted.

Purpose Integrity

Emergency Release Purpose ≠ Automatic Exemption From Mandatory Release

Conditions.

Readiness Assessment Boundary

The assessment boundary defines the conditions covered by the readiness

determination.

Relevant dimensions include:

Candidate Artifact Boundary

Functional Scope Boundary

Requirements Boundary

Quality Boundary

Security Boundary

Dependency Boundary

Configuration Boundary

Platform Boundary

Evidence Boundary

Temporal Boundary

Boundary Integrity

Readiness Established Within a Limited Boundary ≠ Readiness Established for Every Software Use or Environment.

Applicable Release Criteria

RRM identifies the criteria governing readiness for the particular

candidate and release context.

Relevant elements include:

Criterion ID

Criterion Purpose

Criterion Source

Criterion Applicability

Required Evidence

Acceptance Condition

Mandatory or Conditional Status

Exception Eligibility

Assessment Result

Unresolved Condition

Readiness criteria must be identifiable before their satisfaction is asserted.

Criteria Integrity

Release Criteria Documented ≠ Release Criteria Satisfied.

Criteria must not be retroactively weakened merely to convert an unfavorable readiness result into a favorable one.

Requirements Alignment Assessment

RRM evaluates whether relevant findings support alignment between the

proposed release and applicable software requirements.

Relevant elements include:

Requirements Baseline Reference

Release Scope

Implemented Functionality References

Requirements Traceability

Known Requirements Deviations

Unapproved Functionality Findings

Unimplemented Mandatory Requirements

Requirements Evidence

RRM uses SRS and relevant lifecycle evidence rather than independently

redefining approved requirements.

Requirements Alignment Principle

Software Functionality Working ≠ Software Functionality Authorized by

Requirements.

Quality Condition Assessment

RRM evaluates release-relevant quality findings established through SQS.

Relevant elements include:

Quality Requirements Reference

Quality Criteria

Test Coverage Findings

Test Execution Results

Defect Classification

Unresolved Defects

Quality Limitations

Quality Assessment Validity

A favorable quality assessment must remain associated with the candidate and conditions actually evaluated.

Quality Integrity

Quality Tests Passed ≠ All Material Quality Conditions Satisfied.

Security Condition Assessment

RRM evaluates release-relevant security findings established through SSS.

Relevant elements include:

Security Requirements

Security Testing Findings

Vulnerability Status

Exposure Conditions

Security Remediation References

Unresolved Security Findings

Security Restrictions

Security Evidence Validity

Security findings must not be concealed or treated as resolved merely because the release pipeline completed.

Security Integrity

Security Scan Completed ≠ Software Security Conditions Satisfied.

Dependency Condition Assessment

RRM evaluates the release implications of dependency conditions established

through SDS.

Relevant elements include:

Dependency Baseline Reference

Dependency Acceptance State

Critical Dependency Findings

Dependency Compatibility

Dependency Restrictions

Dependency Risk Findings

Unresolved Dependency Conditions

Dependency Change References

RRM does not independently authorize dependency introduction.

Dependency Integrity

Dependency Installed Successfully ≠ Dependency Acceptable for Release.

Configuration Condition Assessment

RRM evaluates relevant configuration findings established through SCFS.

Relevant elements include:

Configuration Baseline ID

Build Identity

Artifact Version Identity

Configuration Consistency

Build Reproducibility Findings

Artifact Equivalence Findings

Configuration Exceptions

Unresolved Configuration Conditions

Configuration Integrity

Configuration Baseline Recorded ≠ Release Candidate Configuration

Verified.

Artifact Integrity Assessment

RRM evaluates whether available artifact identity and integrity findings

remain applicable to the candidate.

Relevant elements include:

Release Candidate ID

Artifact Identity Reference

Build Artifact Reference

Artifact Integrity Evidence

Artifact Modification History

Artifact Comparison Findings

Unresolved Identity Conflicts

RRM consumes relevant RCMM and SCFS findings.

Artifact Integrity Principle

Artifact Integrity Evidence Available ≠ Every Constituent Artifact

Verified.

Test Coverage Relevance

RRM evaluates whether available test coverage is relevant to the release candidate and its applicable requirements and risks.

Relevant elements include:

Tested Requirements

Tested Components

Tested Configurations

Tested Interfaces

Tested Conditions

Untested Areas

Known Coverage Limitations

Candidate Applicability

RRM does not prescribe a universal test coverage percentage.

Coverage Principle

High Test Coverage ≠ Sufficient Coverage of Critical Software Behavior.

Test Result Applicability

RRM must determine whether available test results apply to the candidate being assessed.

Relevant elements include:

Tested Artifact ID

Current Candidate ID

Test Environment

Test Configuration

Test Execution Time

Relevant Dependency State

Material Changes Since Testing

Evidence Applicability

Test Applicability Principle

Tests Passed on a Previous Build ≠ Tests Passed on the Current Release

Candidate.

Unresolved Defect Assessment

RRM preserves release-relevant unresolved defects and their implications.

Relevant elements include:

Defect ID

Affected Component

Severity

Operational Significance

Requirements Impact

Known Workaround

Resolution Status

Release Restriction

Supporting Evidence

A defect may be technically unresolved without automatically prohibiting every release, provided applicable mandatory conditions and exception rules are respected.

Defect Assessment Principle

Defect Classified as Low Severity ≠ Defect Immaterial to Every Release

Context.

Unresolved Vulnerability Assessment

RRM evaluates the release implications of unresolved security

vulnerabilities.

Relevant elements include:

Vulnerability Reference

Affected Artifact

Exposure Context

Severity

Exploitability Findings

Potential Consequences

Mitigation State

Applicable Release Restrictions

Security Assessment Reference

Vulnerability Principle

Vulnerability Documented ≠ Vulnerability Risk Resolved.

Cross-Condition Consistency

RRM evaluates whether findings from different lifecycle domains remain

mutually consistent.

Relevant conflicts may include:

Requirements–Implementation Conflict

Build–Configuration Conflict

Dependency–Security Conflict

Quality–Security Conflict

Artifact–Evidence Conflict

Scope–Testing Conflict

Assessment–Candidate Identity Conflict

Material conflicts must remain visible until appropriately resolved.

Consistency Principle

Individual Assessments Passed ≠ Combined Release Conditions Consistent.

Release Readiness Evidence

RRM may use:

Requirements Traceability Records

Software Creation Records

Software Release Candidate Profile

Dependency Governance Profiles

Configuration Baselines

Build Records

Artifact Integrity Records

Quality Assessment Records

Test Coverage Records

Defect Records

Security Assessment Records

Vulnerability Records

Exception Decisions

Relevant Change Records

Evidence must be relevant to the specific candidate and readiness criteria.

Evidence Provenance

Readiness evidence must preserve:

Evidence ID

Evidence Source

Evidence Origin

Assessment or Collection Method

Referenced Candidate

Referenced Artifact

Relevant Configuration

Evidence Time

Evidence Content

Known Limitations

Version

Evidence Provenance Principle

Readiness Evidence Recorded ≠ Readiness Evidence Authentic or Applicable.

Evidence Sufficiency

Evidence sufficiency concerns whether the available evidence adequately

supports the applicable release readiness conditions.

Relevant dimensions include:

Requirements Coverage

Quality Evidence Coverage

Security Evidence Coverage

Dependency Evidence Coverage

Configuration Evidence Coverage

Artifact Identity Coverage

Evidence Reliability

Evidence Freshness

Evidence Consistency

Assessment Boundary Coverage

Evidence Sufficiency Principle

Large Volume of Test Evidence ≠ Sufficient Release Readiness Evidence.

Evidence Freshness

RRM evaluates whether evidence remains applicable to the current release

candidate and relevant conditions.

Relevant elements include:

Evidence Generation Time

Candidate Creation Time

Candidate Modification Time

Assessment Time

Configuration Changes

Dependency Changes

Relevant Security Changes

Evidence Validity Conditions

Freshness Principle

Evidence Previously Valid ≠ Evidence Currently Applicable.

Missing and Conflicting Evidence

RRM must preserve evidence gaps and conflicts affecting readiness.

Relevant conditions include:

Missing Requirements Evidence

Missing Quality Evidence

Missing Security Evidence

Missing Dependency Evidence

Missing Configuration Evidence

Missing Artifact Identity Evidence

Conflicting Test Results

Contradictory Assessment Findings

Unresolved Evidence Applicability

Evidence Gap Principle

Required Evidence Missing ≠ Applicable Condition Satisfied.

Readiness Condition States

Each applicable readiness condition must retain a distinguishable condition state.

The SLGA™ condition model includes:

Satisfied

Not Satisfied

Unknown

Not Applicable

Exception

A condition classified as Not Applicable requires an established basis for

inapplicability.

An Exception requires an applicable exception basis and must remain distinguishable from Satisfied.

Unknown must not be treated as Satisfied merely because no failure has been observed.

Condition State Integrity

Unknown ≠ Satisfied.

Not Applicable ≠ Not Assessed.

Exception ≠ Satisfied.

No Known Failure ≠ Positive Evidence of Readiness.

Material Finding Assessment

RRM evaluates whether unresolved findings materially affect release

readiness.

Relevant dimensions include:

Affected Requirement

Affected Functionality

Affected Artifact

Operational Criticality

Security Consequence

Quality Consequence

Dependency Consequence

Configuration Consequence

Evidence Uncertainty

Applicable Release Restriction

Materiality must be assessed within the actual release context.

Materiality Principle

Finding Minor in Technical Size ≠ Finding Immaterial to Release Readiness.

Release Readiness Exceptions

RRM governs the readiness implications of exceptions established under

applicable governance rules.

Relevant elements include:

Exception ID

Affected Criterion

Exception Reason

Exception Eligibility

Authority Reference

Exception Scope

Applicable Restrictions

Validity Period

Residual Uncertainty

Required Follow-Up

RRM does not independently create general authority to override mandatory requirements.

Exception Integrity

Exception Approved ≠ Underlying Release Condition Satisfied.

An exception cannot override a non-exceptionable mandatory condition.

Conditional Readiness

Conditional readiness may be established only where applicable rules permit readiness to be qualified by explicit conditions or restrictions.

Relevant elements include:

Conditional Readiness Basis

Applicable Criteria

Satisfied Conditions

Excepted Conditions

Restrictions

Unresolved Limitations

Evidence Basis

Authorization Implications

Conditional readiness must not conceal an unresolved mandatory prohibition.

Conditional Readiness Principle

Conditional Readiness ≠ Unrestricted Release Readiness.

Readiness Classification

RRM may classify readiness states as:

Assessment Not Initiated

Assessment Pending

Criteria Incomplete

Evidence Collection Pending

Evidence-Limited

Conditions Under Review

Readiness Established

Conditional Readiness Established

Readiness Not Established

Mandatory Condition Not Satisfied

Exception Review Required

Material Finding Unresolved

Conflicted

Unknown

Reassessment Required

Readiness Superseded

Assessment Closed

These are release readiness classifications, not release authorization decisions.

Classification Integrity

Readiness Established ≠ Release Authorized.

A readiness determination must be transferred to the appropriate release authorization mechanism.

Readiness Validity Conditions

Every established readiness conclusion must remain associated with its applicable validity conditions.

Relevant elements include:

Release Candidate Identity

Assessment Scope

Artifact Baseline

Configuration Baseline

Dependency Baseline

Requirements Baseline

Quality Assessment References

Security Assessment References

Evidence Applicability

Relevant Time Conditions

Known Restrictions

Validity Principle

Readiness Established Under One Set of Conditions ≠ Readiness Established

Under Materially Different Conditions.

Readiness Reassessment

RRM governs the need to reassess readiness when relevant conditions change.

Reassessment triggers may include:

Candidate Artifact Change

Requirements Change

Configuration Change

Dependency Change

New Quality Finding

New Security Finding

Test Evidence Invalidated

Previously Unknown Condition Identified

Exception Expiration

Assessment Boundary Change

Material Evidence Conflict

Reassessment may be targeted to affected conditions where sufficient evidence establishes that unaffected findings remain applicable.

Reassessment Principle

Readiness Reassessment Triggered ≠ Previous Readiness Automatically

Reconfirmed.

Software Release Readiness Profile

The principal governed output is the:

Software Release Readiness Profile

It establishes the evidence-supported readiness state of an identifiable

release candidate.

The profile preserves:

• Candidate identity

• Assessment identity

• Assessment purpose and scope

• Applicable readiness criteria

• Requirements alignment

• Quality condition state

• Security condition state

• Dependency condition state

• Configuration condition state

• Artifact integrity references

• Evidence sufficiency

• Unresolved findings

• Exceptions and restrictions

• Readiness classification

• Validity conditions

• Reassessment requirements

• Authorization handoff

Minimum Profile Fields

At minimum:

Software Release Readiness Profile ID

Readiness Assessment ID

Release Candidate ID

Software Product ID

Candidate Revision

Assessment Purpose

Assessment Boundary

Applicable Readiness Criteria

Requirements Baseline Reference

Requirements Alignment State

Quality Assessment Reference

Quality Condition State

Security Assessment Reference

Security Condition State

Dependency Baseline Reference

Dependency Condition State

Configuration Baseline Reference

Configuration Condition State

Artifact Integrity Reference

Test Coverage Reference

Unresolved Defect Register

Unresolved Vulnerability Register

Material Finding Register

Evidence Provenance

Evidence Sufficiency

Evidence Freshness

Missing Evidence

Evidence Conflicts

Condition State Register

Exception References

Release Restrictions

Readiness Classification

Validity Conditions

Reassessment Requirement

Authorization Handoff Status

Effective Time

Version

Minimum Implementation Framework

A minimum operational RRM implementation should contain eight governed layers. Layer 1 — Candidate and Assessment Identity

Establish the exact release candidate, assessment identity, purpose, scope,

and boundaries. Layer 2 — Applicable Readiness Criteria

Identify the release conditions, mandatory requirements, acceptance

criteria, and permitted exception rules. Layer 3 — Requirements and Quality Assessment

Evaluate applicable requirements alignment, quality findings, test

coverage, and unresolved defects. Layer 4 — Security, Dependency and Configuration Assessment

Evaluate security findings, dependency conditions, configuration

consistency, and relevant artifact integrity evidence. Layer 5 — Evidence and Condition Assessment

Determine evidence provenance, sufficiency, freshness, applicability,

conflicts, and individual condition states. Layer 6 — Material Findings and Exceptions

Evaluate unresolved findings, materiality, restrictions, and valid

exception conditions. Layer 7 — Readiness Classification and Validity

Establish the readiness determination and preserve the conditions under

which it remains applicable. Layer 8 — Governed Output and Authorization Handoff

Produce the Software Release Readiness Profile and transfer the governed

readiness state to RAM.

Minimum Decision States

At minimum:

Assessment Not Initiated

Candidate Identified

Assessment Boundary Established

Readiness Criteria Identified

Criteria Applicability Pending

Requirements Assessment Pending

Quality Assessment Pending

Security Assessment Pending

Dependency Assessment Pending

Configuration Assessment Pending

Evidence Review Pending

Evidence Sufficient

Evidence Insufficient

Evidence Conflicted

Condition Satisfied

Condition Not Satisfied

Condition Unknown

Condition Not Applicable

Condition Exception

Material Finding Identified

Exception Review Required

Readiness Not Established

Conditional Readiness Established

Readiness Established

Reassessment Required

Readiness Superseded

Authorization Handoff Completed

Unknown

Minimum Evidence Path

Where release readiness cannot be sufficiently established:

Software Release Candidate Profile → Assessment Scope → Applicable Release

Criteria → Requirements Evidence → Quality Evidence → Security Evidence → Dependency Evidence → Configuration Evidence → Artifact Integrity Evidence → Evidence Sufficiency Review → Condition Classification → Missing / Conflicted / Unknown / Not Satisfied Condition → Materiality Assessment → Evidence Acquisition / Corrective Handoff / Exception Review → Targeted Reassessment → Readiness Classification → Software Release Readiness Profile → RAM Authorization Handoff

Missing evidence must not be replaced by assumed readiness.

Minimum Reassessment Framework

Current Software Release Readiness Profile → New Candidate or Lifecycle

Event → Material Change Identification → Candidate Identity Review → Requirements Applicability Review → Quality Evidence Review → Security Evidence Review → Dependency Condition Review → Configuration Condition Review → Evidence Sufficiency Review → Exception Validity Review → Condition State Update → Readiness Reclassification → Historical State Preservation → Profile Update → Authorization Handoff Update

Reassessment must distinguish:

Candidate Changed

Requirements Changed

Quality Conditions Changed

Security Conditions Changed

Dependencies Changed

Configuration Changed

Evidence Changed

Exception Conditions Changed

Previous Readiness Still Supported

Readiness Reassessment Required

Readiness No Longer Established

Governance Requirements

A conforming RRM implementation must satisfy the following requirements. Assessment Identity and Criteria

• Identify the release candidate being assessed

• Establish the readiness assessment identity

• Define assessment purpose

• Define assessment scope

• Preserve assessment boundaries

• Identify applicable release criteria

• Distinguish mandatory and conditional criteria

• Preserve criterion applicability

• Identify permitted exception conditions

• Prevent incomplete criteria from automatically establishing readiness.

Requirements and Quality

• Preserve applicable requirements references

• Evaluate release scope alignment

• Identify material requirements deviations

• Preserve quality assessment references

• Evaluate relevant test coverage

• Preserve unresolved defects

• Assess defect materiality

• Prevent build success from replacing quality assessment

• Prevent passing tests from automatically establishing requirements alignment.

Security, Dependencies and Configuration

• Preserve security assessment references

• Evaluate unresolved security findings

• Preserve relevant vulnerability conditions

• Preserve dependency baseline references

• Evaluate dependency acceptance conditions

• Identify unresolved dependency restrictions

• Preserve configuration baseline references

• Evaluate configuration consistency

• Preserve artifact integrity references

• Prevent successful dependency installation from establishing

• Prevent configuration declarations from replacing configuration

dependency acceptance. evidence.

Evidence Governance

• Preserve readiness evidence provenance

• Preserve evidence applicability to the exact candidate

• Evaluate evidence sufficiency

• Evaluate evidence freshness

• Identify missing evidence

• Preserve conflicting evidence

• Preserve relevant uncertainty

• Prevent historical evidence from automatically applying after

• Prevent evidence volume from replacing evidence relevance

• Prevent absent findings from being represented as positive evidence

material candidate changes. of readiness.

Conditions and Exceptions

Classify applicable conditions as Satisfied, Not Satisfied, Unknown,

Not Applicable, or Exception. 42. Preserve the basis for Not Applicable conditions. 43. Preserve the basis and authority references for exceptions. 44. Identify non-exceptionable mandatory conditions. 45. Preserve unresolved mandatory failures. 46. Assess material findings. 47. Preserve release restrictions. 48. Prevent Unknown from being treated as Satisfied. 49. Prevent Exception from being treated as Satisfied. 50. Prevent conditional readiness from being represented as unrestricted readiness.

Readiness Classification and Continuity

• Establish an evidence-supported readiness classification

• Preserve readiness validity conditions

• Bind readiness to the assessed candidate identity

• Preserve assessment limitations

• Identify material reassessment triggers

• Reassess affected conditions when required

• Preserve historical readiness states

• Preserve superseding readiness conclusions

• Prevent readiness for one candidate from automatically transferring

• Prevent readiness classification from becoming release authorization

to another.

Architecture Boundaries

• Maintain RCMM candidate identification boundaries

• Maintain SRS requirements governance boundaries

• Maintain SDS dependency governance boundaries

• Maintain SCFS configuration governance boundaries

• Maintain SQS quality governance boundaries

• Maintain SSS security governance boundaries

• Maintain RAM release authorization boundaries

• Maintain SCMS change governance boundaries

• Maintain SLCS lifecycle control boundaries

• Produce a Software Release Readiness Profile

Governance Logic

Canonical RRM logic:

Release Candidate Identity → Readiness Assessment Purpose → Assessment

Boundary → Applicable Release Criteria → Requirements Alignment → Quality Condition Assessment → Security Condition Assessment → Dependency Condition Assessment → Configuration Condition Assessment → Artifact Integrity Review → Test Evidence Applicability → Unresolved Findings → Evidence Provenance → Evidence Sufficiency → Evidence Freshness → Condition State Classification → Materiality Assessment → Exception & Restriction Review → Readiness Determination → Validity Condition Establishment → Software Release Readiness Profile → RAM Authorization Handoff

A readiness determination must not override unresolved mandatory conditions or create release authority.

Governance Outputs

RRM may produce:

Software Release Readiness Profile

Readiness Assessment Identity

Assessment Purpose Record

Assessment Boundary

Release Criteria Register

Requirements Alignment Assessment

Quality Condition Assessment

Security Condition Assessment

Dependency Condition Assessment

Configuration Condition Assessment

Artifact Integrity Assessment Reference

Test Coverage Applicability Record

Unresolved Defect Register

Unresolved Vulnerability Register

Material Finding Assessment

Readiness Evidence Register

Evidence Provenance Record

Evidence Sufficiency State

Evidence Freshness Assessment

Missing Evidence Record

Evidence Conflict Record

Condition State Register

Exception Reference

Release Restriction Record

Readiness Classification

Readiness Validity Conditions

Reassessment Requirement

Historical Readiness Reference

Authorization Handoff

Version

Practical Applications Autonomous Software Factories

Evaluating whether AI-generated software candidates satisfy release

requirements rather than relying solely on automated build and test success. Enterprise Release Management

Establishing consistent release readiness conditions across development

teams, software products, and delivery environments. Continuous Delivery Pipelines

Separating pipeline execution success from evidence-supported release

readiness. Security Patch Releases

Assessing whether emergency software updates satisfy applicable release

conditions while preserving unresolved risks and restrictions. Critical Infrastructure Software

Evaluating release readiness where software defects, configuration changes, or security vulnerabilities may affect essential operational functions. AI-Assisted Software Maintenance

Reassessing readiness after AI agents modify existing software,

dependencies, configurations, or implementation behavior.

• Use Case 1 — An AI Agent Produces Software That Passes Every Test but

Contains an Unauthorized Function

An organization authorizes an autonomous AI development system to create a

financial reporting application.

The approved requirements define data collection, calculation, and report

generation functions.

During development, an AI agent introduces an automatic refund function

that was not included in the approved requirements.

Another agent generates tests for the implemented application.

All tests pass.

The automated pipeline reports:

Build Successful — Tests Passed — Release Ready.

The application is submitted for release authorization.

RRM evaluates:

Release Candidate Identity → Requirements Alignment → Test Coverage →

Quality Findings → Evidence Sufficiency → Readiness Classification

The assessment identifies:

• The release candidate is identifiable

• The application builds successfully

• Automated tests pass

• The additional refund function is outside the approved requirements

• Test success establishes that the implemented function behaves as

• Requirements alignment remains unresolved for the proposed release

tested, not that its inclusion was authorized.

The appropriate readiness classification is:

Readiness Not Established — Material Requirements Deviation.

The critical distinctions are:

Testing Success ≠ Requirements Alignment.

Implemented Functionality ≠ Authorized Functionality.

Automated Pipeline Success ≠ Release Readiness.

The result: RRM prevents a known requirements deviation from being

concealed behind successful automated testing.

RRM does not guarantee detection of every hidden function. It requires identified deviations and evidence limitations to remain explicit in the readiness determination.

Use Case 2 — A Critical Security Vulnerability Remains Unresolved

Before Release

A software company prepares a new version of an enterprise application.

The candidate has passed functional tests and satisfies its principal

performance requirements.

A security assessment identifies a vulnerability affecting an externally

accessible interface.

The development team records the vulnerability but does not complete remediation.

The release pipeline nevertheless reports that all required build and functional test stages have completed successfully.

The organization proposes to release the software because the application

is technically functional.

RRM evaluates:

Candidate Identity → Security Findings → Vulnerability Materiality →

Applicable Security Criteria → Evidence Sufficiency → Exception Eligibility → Readiness Classification

The assessment identifies:

• Functional quality conditions are supported

• A material security finding remains unresolved

• Technical functionality does not resolve the security condition

• Applicable release criteria must determine whether release is

• The vulnerability cannot be treated as satisfied merely because it has

prohibited, restricted, or eligible for a legitimate exception. been documented.

Where the applicable mandatory security condition prohibits release, the readiness classification is:

Readiness Not Established — Mandatory Security Condition Not Satisfied.

The critical distinctions are:

Functional Readiness ≠ Security Readiness.

Vulnerability Documented ≠ Vulnerability Resolved.

Exception Requested ≠ Exception Authorized.

The result: RRM preserves the security condition as an independent

release readiness requirement and prevents unrelated technical success from overriding it.

• Use Case 3 — Previously Valid Test Evidence Is Reused After a

Dependency Change

An organization completes release readiness assessment for a software

candidate.

Requirements alignment, quality findings, and security conditions are supported by relevant evidence.

Before authorization, an automated maintenance process updates a critical

external dependency.

The candidate retains the same visible version label.

The development pipeline does not rerun all affected tests.

Instead, it reuses the earlier readiness report.

RRM evaluates:

Original Candidate → Dependency Change → Artifact Applicability → Previous

Evidence → Affected Conditions → Reassessment Requirement → Updated Readiness State

The assessment identifies:

• Previous readiness evidence applied to the earlier candidate state

• A material dependency change occurred

• The unchanged version label does not establish artifact equivalence

• Previous test and security evidence may no longer cover the modified

• Targeted reassessment is required for affected conditions

• Unaffected findings may be retained only where their continued

candidate. applicability is sufficiently supported.

The appropriate classification is:

Reassessment Required — Previous Readiness Not Automatically Applicable.

The critical distinctions are:

Version Equality ≠ Artifact Equivalence.

Evidence Previously Valid ≠ Evidence Currently Applicable.

Previous Readiness ≠ Continuing Readiness.

The result: RRM prevents historical readiness evidence from being

automatically transferred to materially changed software.

Critical Distinctions

Build Success ≠ Release Readiness.

Testing Success ≠ Sufficient Release Verification.

Implemented Behavior ≠ Approved Software Requirements.

Readiness Evidence Available ≠ Readiness Evidence Sufficient.

Release Conditions Assessed ≠ Release Conditions Satisfied.

Unknown Release Condition ≠ Satisfied Release Condition.

Release Exception Approved ≠ Underlying Condition Satisfied.

Release Readiness Established ≠ Release Authorization Granted.

Readiness Established for One Candidate ≠ Readiness Established for

Another Candidate.

Readiness Assessment Recorded ≠ Readiness Assessment Applicable to the

Current Candidate.

Emergency Release Purpose ≠ Automatic Exemption From Mandatory Release

Conditions.

Readiness Established Within a Limited Boundary ≠ Readiness Established for Every Software Use or Environment.

Software Functionality Working ≠ Software Functionality Authorized by

Requirements.

Quality Tests Passed ≠ All Material Quality Conditions Satisfied.

Security Scan Completed ≠ Software Security Conditions Satisfied.

Dependency Installed Successfully ≠ Dependency Acceptable for Release.

Configuration Baseline Recorded ≠ Release Candidate Configuration

Verified.

High Test Coverage ≠ Sufficient Coverage of Critical Software Behavior.

Tests Passed on a Previous Build ≠ Tests Passed on the Current Release

Candidate.

Defect Classified as Low Severity ≠ Defect Immaterial to Every Release

Context.

Vulnerability Documented ≠ Vulnerability Risk Resolved.

Individual Assessments Passed ≠ Combined Release Conditions Consistent.

Readiness Evidence Recorded ≠ Readiness Evidence Authentic or Applicable.

Large Volume of Test Evidence ≠ Sufficient Release Readiness Evidence.

Evidence Previously Valid ≠ Evidence Currently Applicable.

Required Evidence Missing ≠ Applicable Condition Satisfied.

Not Applicable ≠ Not Assessed.

Exception ≠ Satisfied.

No Known Failure ≠ Positive Evidence of Readiness.

Finding Minor in Technical Size ≠ Finding Immaterial to Release Readiness.

Conditional Readiness ≠ Unrestricted Release Readiness.

Readiness Established Under One Set of Conditions ≠ Readiness Established

Under Materially Different Conditions.

Readiness Reassessment Triggered ≠ Previous Readiness Automatically

Reconfirmed.

Cross-Architecture Boundary

RRM may consume governed evidence from:

SRS — Software Requirements Standard

SCS — Software Creation Standard

SDS — Software Dependency Standard

SCFS — Software Configuration Standard

SQS — Software Quality Standard

SSS — Software Security Standard

SCMS — Software Change Management Standard

SLCS — Software Lifecycle Control Standard

Within SRLS, RRM receives the candidate identity and composition established by:

RCMM — Release Candidate Management Module

RRM provides the readiness determination to:

RAM — Release Authorization Module

Relevant decisions and evidence may subsequently be preserved through:

RRMM — Release Record Management Module

Material changes or invalidated release conditions may also become relevant to:

RWM — Release Withdrawal Module

RRM does not replace the substantive governance mechanisms responsible for requirements, quality, security, dependencies, configuration, or software changes.

Its role is to determine whether their relevant findings sufficiently support release readiness for the identified candidate.

Cross-Architecture Mechanism Principle

An operational mechanism may produce inputs for multiple governance architectures, but it belongs only to the architecture whose governed domain includes that mechanism itself. Other architectures govern only its domain-specific relevance or consequences.

RRM may consume software test results, vulnerability assessments, dependency records, configuration evidence, and requirements traceability.

It governs the release readiness implications of those findings.

It does not independently redefine their underlying assessment methods or assume the authority of the mechanisms that produced them.

Canonical Boundary

RRM governs how identifiable software release candidates are assessed against applicable requirements, quality, security, dependency, configuration, artifact integrity, evidence, and release-specific conditions to establish an evidence-supported readiness determination within explicit assessment and validity boundaries.

It does not independently create software requirements, execute testing, remediate vulnerabilities, authorize software release, authorize deployment, or certify that software is free from defects or harmful behavior.

Its principal governed output is the Software Release Readiness Profile.

Canonical Closing Statement

Software release readiness cannot be established merely because

development is complete, a build succeeds, or automated tests report success.

A software candidate may perform its tested functions correctly while containing behavior that was never authorized.

It may satisfy quality requirements while retaining unresolved security vulnerabilities.

It may be supported by extensive test evidence that applies to an earlier build rather than the artifact currently proposed for release.

And it may receive a favorable automated readiness report while material dependency, configuration, or requirements conditions remain unknown.

The absence of a detected failure is not evidence that every applicable release condition has been satisfied.

RRM establishes the methodological boundary between technical results and

evidence-supported release readiness.

It requires the exact candidate to be identified, the applicable criteria

to be established, the relevant lifecycle findings to be assessed, and unresolved conditions to remain visible.

It preserves the distinction between satisfied conditions, unsatisfied

conditions, unknown conditions, legitimate inapplicability, and explicitly governed exceptions.

It also ensures that readiness remains associated with the software artifact and conditions actually assessed.

A readiness conclusion is not permanent merely because it was valid when first established.

When software changes, dependencies change, configurations change, or new evidence emerges, the continued applicability of the readiness conclusion must be reassessed.

RRM establishes a fundamental release governance requirement: software

must not be represented as ready for authorization unless the applicable release conditions are sufficiently supported by relevant evidence and unresolved limitations remain explicitly governed.

This requirement applies regardless of whether software is developed by humans, automated pipelines, AI coding assistants, or autonomous software factories.

The actor may change.

The development process may accelerate.

The volume of automated testing may increase.

The requirement for evidence-supported release readiness remains.

RRM transforms release readiness from a technical success indicator into a

governed assessment state — ensuring that software is not considered ready merely because it works, but because the conditions required to justify its release have been sufficiently established.